

SOFTWARE QUALITY ASSURANCE

SQA Complete Theory & Viva Guide

Every concept behind your CV — testing types, automation, performance, API, accessibility & Agile — with exhaustive viva questions and model answers.

Prepared for: **Abrar Ragib** — Software Quality Assurance Engineer

Scope: **Manual** · **Automation (Selenium/Playwright)** · **JMeter** · **API** · **Accessibility** · **Agile**

Format: **Theory + Mock Viva Q&A for every topic**

A self-study reference mapped directly to the skills, tools and projects on your resume.

Table of Contents

1. SDLC & STLC — the foundation
2. Verification vs Validation & QA/QC/Testing
3. Levels of Testing
4. Types of Testing (Functional & Non-functional)
5. Testing Methods: Black / White / Grey Box
6. Test Case Design Techniques
7. Defect / Bug Life Cycle, Severity & Priority
8. Test Artifacts & Documentation
9. Automation Testing — Selenium (Python)
10. Automation Testing — Playwright
11. Page Object Model & CI/CD Integration
12. Performance & Load Testing — Apache JMeter
13. API Testing — Postman & REST
14. Accessibility Testing & WCAG
15. OCR / ML-System & Cross-Platform Testing
16. Agile, Scrum & Defect Tracking (Jira)
17. Database / SQL Testing
18. Rapid-Fire Viva Question Bank

1. SDLC & STLC — the Foundation

Your resume lists "STLC · SDLC" as core knowledge. Interviewers almost always open here, so master the phases and how the two relate.

1.1 Software Development Life Cycle (SDLC)

The **SDLC** is the structured process a team follows to build software. It defines *what* is built and *in what order*. Classic phases:

1. **Requirement gathering & analysis** — collect business/functional needs, produce the SRS (Software Requirement Specification).
2. **Design** — high-level (architecture) and low-level (module) design, HLD/LLD documents.
3. **Implementation / Coding** — developers write the code.
4. **Testing** — QA validates against requirements (this is where the STLC lives).
5. **Deployment** — release to production.
6. **Maintenance** — bug fixes, enhancements, patches.

Common SDLC models

Model	Idea	Best for
Waterfall	Linear, each phase completes before the next	Fixed, well-known requirements
V-Model	Each dev phase has a matching test phase (verification & validation)	Safety-critical, strict traceability
Iterative / Incremental	Build in repeating cycles, add features each round	Large evolving systems
Agile / Scrum	Short sprints, continuous feedback, working software fast	Changing requirements (your daily reality)
Spiral	Iterative + heavy risk analysis each loop	High-risk, large projects

1.2 Software Testing Life Cycle (STLC)

The **STLC** is the testing-specific sub-process that runs inside/alongside the SDLC. Phases:

1. **Requirement analysis** — study requirements from a testability view; identify what to test.
2. **Test planning** — the QA lead defines scope, strategy, effort, tools, and the *test plan* document.
3. **Test case design (development)** — write test cases, test scripts, and prepare test data.
4. **Test environment setup** — configure hardware/software/network needed to run tests.

5. **Test execution** — run cases, log results, report and re-test defects.
6. **Test cycle closure** — evaluate exit criteria, produce test summary report, capture lessons learned.

Entry & Exit criteria: each STLC phase has entry criteria (what must exist to start) and exit criteria (what must be true to finish). E.g., test execution's exit criterion may be "all critical/high defects closed and 95% test cases passed."

Viva — SDLC & STLC

Q What is the difference between SDLC and STLC?

A SDLC is the whole software development process (requirements → maintenance); STLC is the testing subset that focuses only on the QA phases (test planning → closure). STLC is triggered inside the testing phase of SDLC but its analysis begins as early as the requirement phase.

Q Where does QA involvement start in the SDLC?

A As early as the requirement analysis phase. Reviewing requirements early ("shift-left") catches ambiguous/testable-but-incomplete requirements when they are cheapest to fix.

Q What is the V-model and why is it relevant to QA?

A It maps each development phase to a corresponding testing phase — requirements↔acceptance testing, HLD↔system testing, LLD↔integration testing, code↔unit testing. It emphasizes verification and validation running in parallel rather than testing only at the end.

Q What are entry and exit criteria?

A Entry criteria are the preconditions needed to begin a phase (e.g., approved test cases, ready environment); exit criteria are the conditions to declare it complete (e.g., defined pass rate, no open critical bugs).

Q Which SDLC model do you use day-to-day?

A Agile/Scrum — I work in sprints, testing continuously as features are delivered, attending stand-ups, sprint planning, and retrospectives, and regression-testing every increment.

2. Verification vs Validation & QA / QC / Testing

2.1 Verification vs Validation

	Verification	Validation
Question	"Are we building the product right?"	"Are we building the right product?"
Focus	Process & documents	Actual working product
Techniques	Reviews, walkthroughs, inspections, static analysis	Functional & non-functional testing (dynamic)
Code executed?	No (static)	Yes (dynamic)

2.2 QA vs QC vs Testing

- **Quality Assurance (QA)** — *process-oriented*, proactive, prevents defects (standards, audits, process improvement).
- **Quality Control (QC)** — *product-oriented*, reactive, finds defects in the built product (reviews, testing).
- **Testing** — the activity of executing software to find defects; a subset of QC.

Viva — Verification/Validation & QA/QC

Q Give a one-line memory hook for verification vs validation.

A Verification = building the product *right* (static, no execution). Validation = building the *right* product (dynamic, execution).

Q Is testing part of QA or QC?

A Testing is part of QC. QA is the broader, preventive, process-focused discipline; QC (which includes testing) is the corrective, product-focused one.

Q Give an example of verification you have done.

A Reviewing requirement documents and test cases before execution, and doing peer reviews of bug reports/release notes — all static, no code executed.

Q Static vs dynamic testing?

A Static testing examines artifacts without running code (reviews, static analysis). Dynamic testing runs the software with inputs and observes behavior.

3. Levels of Testing

The four classic levels, moving from smallest unit to full acceptance.

Level	What is tested	Who	Example on your projects
Unit testing	Individual functions/ methods in isolation	Developers (sometimes QA)	A single OCR text-cleaning function
Integration testing	Interaction between modules/services	Developers & QA	Upload flow → OCR engine → DOCX export
System testing	The complete integrated system vs requirements	QA	End-to-end OCR portal on Windows/Linux/Web/Android
Acceptance testing (UAT)	Business/user readiness	Client / end users	Government stakeholders signing off before rollout

Integration approaches

- **Big Bang** — integrate everything at once, then test. Simple but hard to isolate failures.
- **Top-down** — test high-level modules first, use **stubs** for lower ones.
- **Bottom-up** — test low-level modules first, use **drivers** for higher ones.
- **Sandwich/Hybrid** — combine top-down and bottom-up.

Viva — Levels of Testing

Q Difference between stub and driver?

A A **stub** is a dummy called module that stands in for a not-yet-ready lower module (top-down). A **driver** is a dummy calling module that invokes a lower module (bottom-up).

Q Who performs acceptance testing?

A The client or end users (UAT), sometimes with QA facilitation. On government projects, stakeholders validate against contractual/business requirements before national rollout.

Q Alpha vs Beta testing?

A Alpha is done in-house by internal testers/employees before release; Beta is done by real external users in their own environment before general availability.

Q What is smoke's relation to levels?

A Smoke is not a level; it is a shallow build-verification test typically run at the system level to confirm a build is stable enough to test further.

4. Types of Testing

Your CV lists: Manual · Functional · Regression · Smoke · Sanity · API · Performance · Load · Accessibility · Cross-Platform. Here is each, precisely.

4.1 Functional types

- **Functional testing** — verifies each feature behaves per requirements (inputs → expected outputs). Black-box in nature.
- **Smoke testing** — "build verification"; a quick, wide, shallow check that critical paths work before deeper testing. If smoke fails, the build is rejected.
- **Sanity testing** — narrow & deep; after a minor change/bug fix, confirm that specific functionality now works. Usually unscripted.
- **Regression testing** — re-run existing tests after a change to ensure nothing previously working has broken. Prime candidate for automation (you cut regression 35%).
- **Re-testing (confirmation)** — re-execute the exact failed test after a fix to confirm the defect is resolved. (Different from regression!)
- **Exploratory testing** — simultaneous learning, test design and execution without predefined scripts; great for finding edge cases.
- **Ad-hoc testing** — informal, no documentation, relies on tester intuition.

4.2 Non-functional types

- **Performance testing** — how fast/stable/scalable under a given workload (response time, throughput).
- **Load testing** — behavior under expected peak load (e.g., 10,000+ concurrent users on the BSCIC platform).
- **Stress testing** — push beyond limits to find the breaking point and how it recovers.
- **Scalability testing** — can it grow (users/data) while maintaining performance?
- **Accessibility testing** — usable by people with disabilities (WCAG, screen readers).
- **Compatibility / Cross-platform testing** — works across OS, browsers, devices (Windows/Linux/Web/Android).
- **Usability testing** — ease of use for real users.
- **Security testing** — resistance to threats (OWASP Top 10).

Smoke vs Sanity — the classic trap: Smoke = *wide & shallow* on a new build (is it stable at all?). Sanity = *narrow & deep* after a fix (does this specific area work now?). Smoke is usually scripted; sanity usually is not.

Viva — Types of Testing

Q Regression vs re-testing?

A Re-testing confirms a specific fixed defect is gone (same test, same data). Regression checks that the fix/new code didn't break unrelated existing functionality (broad set of tests). Re-testing has higher priority; you re-test first, then regress.

Q When would you automate regression instead of running it manually?

A When the test suite is stable, repetitive, and run every release. I built Selenium (Python) automation for regression which cut manual effort ~35% and freed the team for exploratory testing.

Q Smoke vs sanity — quick definition?

A Smoke = broad, shallow, on a fresh build to decide if it's testable. Sanity = narrow, deep, after a small change to verify that area.

Q What is exploratory testing and when is it valuable?

A Unscripted testing where you design and run tests on the fly using domain knowledge. Valuable for new/complex features (like AI OCR outputs) where scripted cases can't anticipate every edge case.

Q Load vs stress vs scalability?

A Load = expected peak workload; Stress = beyond capacity to find the break point & recovery; Scalability = ability to handle growth in users/data while holding performance.

Q Functional vs non-functional testing?

A Functional checks *what* the system does (features vs requirements). Non-functional checks *how well* it does it (performance, security, usability, accessibility).

5. Testing Methods: Black / White / Grey Box

Method	Knowledge of internals	Who typically	Techniques
Black box	None — tests via UI/inputs only	QA testers	BVA, equivalence partitioning, decision tables
White box	Full — sees the source code	Developers	Statement, branch, path coverage
Grey box	Partial — some internal knowledge (DB schema, APIs)	QA with dev knowledge	Matrix testing, DB validation with UI actions

White-box coverage types

- **Statement coverage** — every line executed at least once.
- **Branch/decision coverage** — every true/false branch taken.
- **Path coverage** — every independent path executed (strongest, most costly).

Viva — Testing Methods

Q As a QA, which do you mostly do?

A Primarily black-box and grey-box — I test through the UI/API against requirements, and use grey-box knowledge (DB schema, API contracts) to verify data end-to-end, e.g., checking that OCR output stored in DB matches what the UI displays.

Q Statement vs branch coverage?

A 100% statement coverage means every line ran, but not every decision outcome. Branch coverage requires each decision to take both true and false paths, so it's stronger — you can hit 100% statements without 100% branches.

Q Give a grey-box example from your work.

A Validating an OCR platform: I upload a document through the UI (black-box), then query the database to confirm extracted text/records were stored correctly (internal knowledge) — a grey-box approach.

6. Test Case Design Techniques

These techniques reduce a huge input space to a small, high-coverage set of tests — the heart of good manual testing.

6.1 Equivalence Partitioning (EP)

Divide input data into **partitions (classes)** that should be treated the same; test one value per partition. If an "age" field accepts 18-60, partitions are: below (<18), valid (18-60), above (>60). One test each covers all values in that class.

6.2 Boundary Value Analysis (BVA)

Bugs cluster at boundaries. For a range 18-60 test: **17, 18, 19** (lower boundary) and **59, 60, 61** (upper boundary). BVA complements EP.

6.3 Decision Table Testing

For combinations of conditions and their resulting actions. Rows = conditions/actions, columns = rules. Ideal for business logic (e.g., discount = f(member, cart value)).

Condition	R1	R2	R3	R4
Is member?	Y	Y	N	N
Cart > 5000?	Y	N	Y	N
Action: Discount	15%	5%	10%	0%

6.4 State Transition Testing

Model the system as states and events causing transitions (e.g., login: LoggedOut → LoggedIn → Locked after 3 fails). Test valid and invalid transitions.

6.5 Error Guessing & Use-Case Testing

Error guessing uses experience to predict likely faults (empty fields, special characters, huge files). **Use-case testing** derives tests from end-to-end user scenarios.

Viva — Test Design

Q EP vs BVA?

A EP groups inputs into classes and tests one per class to reduce test count; BVA tests the edges of those classes (min-1, min, min+1, max-1, max, max+1) because defects concentrate at boundaries. They're used together.

Q For a password field 8-16 chars, give BVA values.

A 7 (invalid), 8 (valid min), 9, 15, 16 (valid max), 17 (invalid) — plus empty and null as extra edge cases.

Q When do you use a decision table?

A When output depends on combinations of multiple conditions — it guarantees every rule combination is covered and exposes missing/contradictory business rules.

Q What technique fits a multi-step workflow with states?

A State transition testing — I map states/events (e.g., order: Draft→Submitted→Approved→Shipped) and test both valid transitions and forbidden ones.

7. Defect Life Cycle, Severity & Priority

7.1 Bug (Defect) Life Cycle

Typical states in a tracker like Jira:

1. **New** — defect logged.
2. **Assigned** — given to a developer.
3. **Open / In Progress** — developer working on it.
4. **Fixed** — developer resolved it.
5. **Retest / Ready for QA** — back to QA.
6. **Verified / Closed** — QA confirms fix; or **Reopened** if still failing.

Non-happy paths: **Duplicate**, **Rejected** (not a bug), **Deferred** (valid but fixed later), **Cannot reproduce** (why your 98% reproduction accuracy matters!), **Won't fix**.

7.2 Severity vs Priority

	Severity	Priority
Defines	Technical impact on the system	Urgency of fixing (business)
Set by	QA / tester	PO / manager (with QA input)
Example — High/Low	App crash on payment (High severity)	Fix now — blocks checkout (High priority)
Example — Low/High	Company logo misspelled (Low severity)	Fix now — visible to all (High priority)
Example — High/Low	Crash in a rarely used admin export (High severity)	Low priority — few users, workaround exists

7.3 A good bug report

Contains: **ID/title**, **environment**, **steps to reproduce**, **expected vs actual result**, **severity/priority**, **screenshots/logs/video**, and reproducibility. Clear reports reduce re-open rates and speed QA-Dev handoff.

Viva — Defects

Q Severity vs priority — give a high-severity/low-priority example.

A A crash in a feature almost nobody uses (e.g., a rarely accessed report export) is high severity (system crashes) but low priority (minimal business impact, workaround exists).

Q Low-severity/high-priority example?

A A typo in the company name on the homepage — cosmetically minor (low severity) but embarrassing and public, so it must be fixed immediately (high priority).

Q A developer marks your bug "Cannot Reproduce." What do you do?

A I reproduce it again on the exact environment, capture detailed steps, logs, video and build/version, then attach them and reassign. Maintaining high, well-documented reproduction accuracy (I sustain ~98%) prevents this from happening in the first place.

Q Difference between error, defect, bug, and failure?

A Error = human mistake in code; defect/bug = the resulting flaw in the software; failure = the defect manifesting during execution as wrong behavior. Error → defect → failure.

Q What is a deferred bug?

A A valid defect whose fix is postponed to a later release, usually because it's low priority/severity and the current release timeline doesn't justify fixing it now.

8. Test Artifacts & Documentation

Your CV highlights authoring test cases, bug reports and release notes. Know the documents cold.

- **Test Plan** — master document: scope, objectives, strategy, resources, schedule, risks, entry/exit criteria.
- **Test Scenario** — high-level "what to test" (e.g., "verify login").
- **Test Case** — detailed steps: ID, precondition, steps, test data, expected result, actual result, status.
- **Test Script** — the automated version of a test case (code).
- **RTM (Requirement Traceability Matrix)** — maps requirements ↔ test cases to prove coverage.
- **Test Summary Report** — results, pass/fail metrics, defect stats at cycle closure.
- **Release Notes** — what changed/fixed/known-issues for a release.

Sample test case format

Field	Example
Test Case ID	TC_LOGIN_005
Precondition	User is registered & on login page
Steps	1. Enter valid email 2. Enter wrong password 3. Click Login
Test Data	email=user@x.com, pwd=Wrong123
Expected	Error "Invalid credentials"; no session created
Actual / Status	As expected / Pass

Viva — Artifacts

Q Test scenario vs test case?

A A scenario is a high-level idea of what to test ("verify checkout"); a test case is the detailed, step-by-step instance with data and expected results. One scenario yields many test cases.

Q What is an RTM and why does it matter?

A The Requirement Traceability Matrix links each requirement to its test cases (and defects). It proves 100% coverage, finds untested requirements, and supports audit readiness — important on government/compliance projects.

Q What goes into a test plan?

A Scope & objectives, features in/out of scope, test strategy, environment, tools, roles, schedule, entry/exit & suspension/resumption criteria, risks and mitigations.

9. Automation Testing — Selenium (Python)

Selenium (Python) is your primary automation stack. Expect deep, practical questions here.

9.1 Selenium components

- **Selenium WebDriver** — the API that drives a real browser via its native driver (ChromeDriver, GeckoDriver).
- **Selenium IDE** — record-and-playback browser extension for quick scripts.
- **Selenium Grid** — run tests in parallel across multiple machines/browsers.

9.2 Locators (most-asked topic)

Priority order for stability: **ID > Name > CSS Selector > XPath**. Prefer ID (fastest, unique). Use XPath when you need to traverse relationships or match text.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.get("https://example.com/login")

# Locators
driver.find_element(By.ID, "username").send_keys("abrar")
driver.find_element(By.CSS_SELECTOR, "input[name='password']").send_keys("secret")
driver.find_element(By.XPATH, "//button[text()='Log in']").click()

# Explicit wait (best practice)
wait = WebDriverWait(driver, 10)
dashboard = wait.until(EC.visibility_of_element_located((By.ID, "dashboard")))
assert dashboard.is_displayed()
driver.quit()
```

9.3 Waits — implicit vs explicit vs fluent

- **Implicit wait** — global; polls the DOM up to N seconds for any element. Set once.
- **Explicit wait** — waits for a *specific condition* on a specific element (preferred, precise).
- **Fluent wait** — explicit wait with custom polling interval and ignored exceptions.

Avoid `time.sleep()` — it's a hard, blind delay that makes suites slow and flaky. Use explicit waits tied to conditions instead.

9.4 Common challenges

- **Dynamic elements** — use stable attributes or relative XPath with `contains()`.

- **Frames & windows** — `switch_to.frame()`, `switch_to.window()`.
- **Alerts** — `switch_to.alert.accept()`.
- **Flaky tests** — caused by timing/order/data; fix with explicit waits, isolated test data, retries.

Viva — Selenium

Q Which locator is most reliable and why?

A ID — it's meant to be unique and is the fastest to resolve. When IDs aren't available I use CSS selectors (faster than XPath) and fall back to XPath for text matching or DOM traversal.

Q Implicit vs explicit wait — can you mix them?

A Explicit waits for a specific condition on one element; implicit sets a global polling timeout for all element lookups. Mixing them is discouraged because their waits can compound unpredictably. I prefer explicit waits everywhere.

Q Why avoid `time.sleep()`?

A It always waits the full duration even if the element is ready sooner (slow) and may still be too short under load (flaky). Explicit waits poll and return as soon as the condition is met.

Q How do you handle a flaky automated test?

A Diagnose the cause — usually timing (replace sleeps with explicit waits), shared/dirty test data (isolate & reset data), or element instability (better locators). Add controlled retries only after fixing the root cause.

Q How did AI assist your automation?

A I used AI assistants (Claude Cowork) to generate and refactor Selenium scripts, suggest robust locators, and scaffold test data — accelerating authoring and helping cut manual regression by ~35%, while I reviewed and validated every script.

Q Difference between `find_element` and `find_elements`?

A `find_element` returns the first match and throws `NoSuchElementException` if none exist; `find_elements` returns a list (empty if none) and never throws — useful for presence checks and counting.

10. Automation Testing — Playwright

You used Playwright for API endpoint testing and UI on HRM. Know why teams pick it over Selenium.

10.1 Why Playwright

- **Auto-waiting** — waits for elements to be actionable automatically, drastically reducing flakiness.
- **Multi-browser** — Chromium, Firefox, WebKit with one API.
- **Built-in API testing** — `request` fixture tests REST endpoints without a browser.
- **Powerful tooling** — tracing, video, network interception, codegen.

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()
    page.goto("https://example.com/login")
    page.fill("#username", "abrar")
    page.fill("#password", "secret")
    page.click("text=Log in")
    page.wait_for_selector("#dashboard") # auto-waits by default
    assert page.is_visible("#dashboard")

# API testing without a browser
resp = page.request.get("https://api.example.com/health")
assert resp.status == 200
browser.close()
```

Viva — Playwright

Q Playwright vs Selenium?

A Playwright auto-waits for actionability (less flaky), supports Chromium/Firefox/WebKit natively, has built-in API testing, tracing and network interception, and runs faster. Selenium has a larger ecosystem, more language bindings, and is the long-standing industry standard. I use both depending on the project.

Q How do you test an API with Playwright?

A Using the request context/fixture (e.g., `page.request.get/post`) to hit endpoints directly, then assert on status codes and JSON bodies — no browser needed, which is fast and stable.

Q What is auto-waiting?

A Before acting, Playwright waits until the element is attached, visible, stable and enabled, so I rarely need explicit waits — this is its biggest flakiness reducer versus Selenium.

11. Page Object Model & CI/CD Integration

11.1 Page Object Model (POM)

A design pattern where each page/screen is a class that encapsulates its **locators** and **actions**. Tests call page methods instead of raw locators — improving reuse and making the suite resilient to UI changes (fix a locator in one place).

```
# pages/login_page.py
from selenium.webdriver.common.by import By

class LoginPage:
    URL = "https://example.com/login"
    USERNAME = (By.ID, "username")
    PASSWORD = (By.ID, "password")
    SUBMIT = (By.XPATH, "//button[text()='Log in']")

    def __init__(self, driver):
        self.driver = driver

    def load(self):
        self.driver.get(self.URL)

    def login(self, user, pwd):
        self.driver.find_element(*self.USERNAME).send_keys(user)
        self.driver.find_element(*self.PASSWORD).send_keys(pwd)
        self.driver.find_element(*self.SUBMIT).click()

# tests/test_login.py
def test_valid_login(driver):
    page = LoginPage(driver)
    page.load()
    page.login("abrar", "secret")
    assert "dashboard" in driver.current_url
```

11.2 CI/CD Integration

CI (Continuous Integration): every code push triggers an automated build + test run. **CD (Continuous Delivery/Deployment)**: passing builds are auto-prepared for or pushed to production. Tests run in pipelines (GitHub Actions, Jenkins, GitLab CI).

```
# .github/workflows/tests.yml
name: QA Automation
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12' }
      - run: pip install -r requirements.txt
      - run: pytest --maxfail=1 --disable-warnings -q
```

Viva — POM & CI/CD

Q What problem does POM solve?

A It removes duplicated locators/steps scattered across tests. When the UI changes, I update one page class instead of dozens of tests — cutting maintenance cost and flakiness.

Q Where do assertions live in POM — page class or test?

A Assertions belong in the test layer; page classes should only expose actions and state. This keeps page objects reusable and tests readable.

Q What triggers your automation in CI?

A Pushes and pull requests trigger the pipeline to install dependencies and run the pytest suite; a failing test blocks the merge, giving fast feedback before code reaches QA/production.

Q CI vs CD?

A CI merges and tests code frequently and automatically; CD extends that by automatically delivering (Continuous Delivery = ready to release) or deploying (Continuous Deployment = auto to prod) every green build.

12. Performance & Load Testing — Apache JMeter

You ran JMeter load tests simulating 10,000+ concurrent users. Know its architecture and metrics.

12.1 Key JMeter elements

- **Test Plan** — root container.
- **Thread Group** — virtual users: *number of threads* (users), *ramp-up period* (time to start all users), *loop count*.
- **Samplers** — the requests (HTTP Request, JDBC, FTP).
- **Listeners** — collect/visualize results (Summary Report, Aggregate Report, View Results Tree).
- **Timers** — pauses between requests (realistic think time).
- **Assertions** — validate responses (Response Assertion, Duration Assertion).
- **Config Elements** — HTTP Defaults, CSV Data Set (data-driven), Cookie Manager.

12.2 Key metrics

- **Throughput** — requests/second the server handles.
- **Response time / Latency** — how long a request takes (avg, median, 90th/95th/99th percentile).
- **Error %** — failed requests under load.
- **Concurrent users** — simultaneous virtual users (your 10,000+).

Percentiles beat averages: the 90th percentile ("90% of users saw response $\leq X$ ") reveals tail latency that a mean hides. Always report percentiles for SLAs.

12.3 Ramp-up & realistic load

Ramp-up gradually adds users so you don't hammer the server instantly (unrealistic). Add timers for think time so the profile mimics real usage. For CI, run JMeter headless (`jmeter -n -t plan.jmx -l results.jtl`).

Viva — JMeter

Q What is ramp-up period?

A The time JMeter takes to start all threads (users). With 1000 users and 100s ramp-up, it adds ~10 users/second — a realistic gradual load instead of an instant spike.

Q How did you simulate 10,000 concurrent users?

A On the BSCIC e-commerce platform I configured Thread Groups with a high thread count and staged ramp-up, used CSV Data Sets for varied inputs, added timers for think time, and analyzed throughput, response-time percentiles and error rate to surface bottlenecks before national rollout.

Q Why report percentiles instead of average response time?

A Averages hide outliers. The 95th/99th percentile shows the experience of the slowest users, which is what SLAs and real perception depend on.

Q What is a JMeter listener and which do you use?

A Listeners collect and display results. Aggregate/Summary Report for metrics; View Results Tree for debugging individual requests. In CI I disable heavy GUI listeners and analyze the .jtl file to keep runs light.

Q How do you correlate dynamic values (tokens) in JMeter?

A Extract them from responses with a post-processor (Regex Extractor or JSON Extractor) into a variable, then reference that variable in later requests — essential for auth tokens and CSRF values.

13. API Testing — Postman & REST

13.1 REST fundamentals

REST uses HTTP methods on resources identified by URLs, is stateless, and typically exchanges JSON.

Method	Purpose	Idempotent?
GET	Read a resource	Yes
POST	Create a resource	No
PUT	Replace a resource fully	Yes
PATCH	Update partially	No (generally)
DELETE	Remove a resource	Yes

13.2 HTTP status codes

Range	Meaning	Examples
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved, 304 Not Modified
4xx	Client error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
5xx	Server error	500 Internal, 502 Bad Gateway, 503 Unavailable

13.3 What to validate in API testing

- **Status code** is correct for the scenario.
- **Response body** — schema, field values, data types.
- **Headers** — content-type, auth, caching.
- **Response time** is within limits.
- **Negative cases** — invalid payloads, missing auth, wrong methods.
- **Auth** — API keys, Bearer/JWT, OAuth flows.

Postman test script & Python requests

```
// Postman "Tests" tab
pm.test("status is 200", () => pm.response.to.have.status(200));
pm.test("has token", () => {
  const json = pm.response.json();
  pm.expect(json).to.have.property("token");
  pm.environment.set("authToken", json.token); // chain to next request
});
```

```
import requests

def test_create_user():
    payload = {"name": "Abrar", "email": "a@x.com"}
    r = requests.post("https://api.example.com/users", json=payload, timeout=5)
    assert r.status_code == 201
    body = r.json()
    assert body["email"] == payload["email"]
    assert "id" in body
```

Viva — API / REST

Q Difference between PUT and PATCH?

A PUT replaces the entire resource (send the full object); PATCH updates only the fields provided. PUT is idempotent; PATCH generally isn't.

Q What does idempotent mean?

A Making the same request multiple times has the same effect as making it once. GET, PUT, DELETE are idempotent; POST is not (it creates a new resource each call).

Q 401 vs 403?

A 401 Unauthorized = not authenticated (no/invalid credentials); 403 Forbidden = authenticated but lacks permission for that resource.

Q How do you chain API requests in Postman?

A In the Tests tab I extract a value (e.g., token or created id) with `pm.environment.set()`, then reference `{{authToken}}` in the next request's header/URL — enabling end-to-end workflow tests.

Q What is the difference between authentication and authorization?

A Authentication verifies *who* you are (login/credentials); authorization determines *what* you're allowed to do (permissions/roles). 401 vs 403 map to these respectively.

Q How would you test file upload + OCR extraction APIs?

A Send multipart POST with valid/invalid/corrupt/oversized files, assert status and returned extracted-text schema, verify accuracy against known ground truth, and cover negatives (wrong format, empty file, missing auth). This mirrors my Fin Doc / OCR work.

14. Accessibility Testing & WCAG

You did end-to-end SQA for the "Screen Reader ALO" app and validated government accessibility systems against compliance standards.

14.1 WCAG & the POUR principles

WCAG (Web Content Accessibility Guidelines) organizes success criteria under four principles — **POUR**:

- **Perceivable** — info must be presentable to all senses (text alternatives/alt text, captions, contrast).
- **Operable** — UI must be usable via keyboard, enough time, no seizure-inducing content.
- **Understandable** — readable, predictable, input assistance (clear errors).
- **Robust** — works with assistive technologies (valid markup, ARIA).

Conformance levels: **A** (minimum), **AA** (target for most laws/policies), **AAA** (highest).

14.2 What accessibility testing checks

- **Keyboard navigation** — everything reachable/operable without a mouse; visible focus order.
- **Screen reader** — content announced correctly (NVDA, JAWS, TalkBack, VoiceOver).
- **Color contrast** — text vs background meets ratio (4.5:1 for normal text at AA).
- **Alt text** for images; labels for form fields.
- **ARIA roles/landmarks** for dynamic content.

Tools: **axe**, **WAVE**, Lighthouse, plus manual screen-reader testing (automation catches only ~30-40%; manual is essential).

Viva — Accessibility

Q What is WCAG and which level do most projects target?

A WCAG is the international accessibility standard built on the POUR principles (Perceivable, Operable, Understandable, Robust). Most legal/government requirements target level AA.

Q Can accessibility be fully automated?

A No. Automated tools (axe, WAVE, Lighthouse) catch roughly a third of issues — things like missing alt text or low contrast. Judgment-based checks (logical reading order, meaningful alt text, screen-reader experience) require manual testing, which I did for the ALO screen reader.

Q How did you test the ALO screen reader?

A End-to-end across Windows and Android: verifying WCAG-aligned functionality, keyboard/gesture navigation, correct element announcements, focus order, and that all interactive content was operable and perceivable to visually impaired users.

Q What is ARIA?

A Accessible Rich Internet Applications — a set of attributes (roles, states, properties) that make dynamic/custom widgets understandable to assistive tech when native HTML semantics aren't enough.

15. OCR / ML-System & Cross-Platform Testing

A distinctive part of your CV: testing AI/OCR systems (Bangla OCR, Kagoj.ai, Fin Doc) and cross-platform delivery.

15.1 Testing OCR / ML outputs

ML outputs are **probabilistic**, not deterministic — so exact-match assertions rarely work. Approaches:

- **Ground-truth comparison** — compare extracted text against a verified reference and compute accuracy metrics.
- **CRR (Character Recognition Rate)** / WER (Word Error Rate) — measure how many characters/words match.
- **Layout preservation** — verify multi-column newspaper images convert to correctly ordered, editable DOCX.
- **Edge cases** — skewed scans, low resolution, mixed fonts, handwriting, noise, mixed languages.
- **Tolerance thresholds** — pass if accuracy $\geq$ agreed threshold (e.g., CRR $\geq$ 98%) rather than exact match.

15.2 Cross-platform / compatibility testing

Verify consistent behavior across **OS** (Windows/Linux/Android), **browsers**, **screen sizes**, and **integrations** (MS Word / LibreOffice plugins). Prioritize a device/OS matrix by user analytics; use real devices for critical paths and emulators for breadth.

Viva — OCR / ML & Cross-Platform

Q How do you test something that doesn't have a single "correct" output, like OCR?

A I compare output against curated ground-truth data and measure accuracy (CRR/WER) against an agreed threshold instead of exact match. I focus on edge cases — skew, noise, low-res, multi-column layouts, mixed scripts — and collaborate with ML engineers to interpret failures.

Q What is CRR?

A Character Recognition Rate — the percentage of characters the OCR engine recognized correctly versus the ground truth; a core accuracy metric I validated on the national OCR portal.

Q How do you decide your cross-platform test matrix?

A By risk and real usage: pick OS/browser/device combinations from user analytics, cover the highest-traffic and highest-risk paths on real devices, and use emulators/VMs for breadth. For OCR I also tested the MS Word/LibreOffice plugins on Windows, Linux, Web and Android.

Q Why is testing AI features harder than deterministic features?

A Because outputs vary with input quality and model changes, so tests must use tolerance thresholds, statistical accuracy metrics and large edge-case datasets rather than fixed expected values, and must be re-baselined when the model is retrained.

16. Agile, Scrum & Defect Tracking (Jira)

16.1 Scrum essentials

- **Sprint** — fixed timebox (1–4 weeks) delivering a potentially shippable increment.
- **Roles** — Product Owner (backlog/priorities), Scrum Master (facilitator), Development Team (incl. QA).
- **Ceremonies** — Sprint Planning, Daily Stand-up, Sprint Review (demo), Retrospective.
- **Artifacts** — Product Backlog, Sprint Backlog, Increment.
- **Definition of Done (DoD)** — shared checklist (code reviewed, tested, no open critical bugs) that says a story is truly complete.

16.2 QA in Agile

QA is embedded, not a separate gate: refine stories for testability, write acceptance criteria, test within the sprint, automate regression continuously, and report status daily. "Shift-left" testing means testing as early as possible.

16.3 Jira workflow

Issues (Story, Bug, Task) move through a board (To Do → In Progress → In Review → Done). Bugs carry severity/priority, steps, environment, and links to the failing story. Confluence stores test plans/notes; dashboards track defect trends.

Viva — Agile / Jira

Q What are acceptance criteria and who writes them?

A Conditions a story must satisfy to be accepted, usually written by the Product Owner and refined with the team/QA. They drive my test cases and define "pass."

Q What is the Definition of Done?

A A team-agreed checklist every story must meet to be "done" — e.g., code reviewed, unit + QA tested, acceptance criteria met, no open critical defects, docs updated. It prevents work leaking between sprints.

Q What happens in a sprint retrospective?

A The team reflects on what went well, what didn't, and agrees concrete improvements for the next sprint — a continuous-improvement loop.

Q How do you handle a bug found late in the sprint?

A Log it in Jira with severity/priority and evidence, discuss with the PO/team whether it blocks the increment; critical/high bugs are fixed and re-tested within the sprint, lower ones may be prioritized into the backlog or deferred.

Q Story vs task vs bug in Jira?

A A story is a user-facing requirement, a task is a piece of work (often technical) that isn't user-facing, and a bug is a defect in existing functionality. All flow through the board but are reported/measured differently.

17. Database / SQL Testing

You list MySQL and used SQL to automate reporting; QA often validates data integrity behind the UI.

Database testing verifies that data is stored, retrieved and updated correctly, checking integrity, constraints, and that UI actions produce the right DB state.

- **CRUD validation** — insert/read/update/delete via UI, verify with SQL.
- **Data integrity** — primary/foreign keys, NOT NULL, UNIQUE constraints.
- **Data mapping** — UI field ↔ correct DB column.
- **Joins/aggregations** for report correctness.

```
-- Verify a UI-created order actually persisted correctly
SELECT o.id, o.status, u.email, o.total
FROM orders o
JOIN users u ON u.id = o.user_id
WHERE o.id = 1024;

-- Check for orphaned records (integrity)
SELECT * FROM orders WHERE user_id NOT IN (SELECT id FROM users);
```

Viva — SQL / DB

Q What is a JOIN and name the main types?

A A JOIN combines rows from related tables on a key. INNER (only matching rows), LEFT (all left + matching right), RIGHT (all right + matching left), FULL (all rows from both).

Q Primary key vs foreign key?

A A primary key uniquely identifies each row in a table (unique, not null). A foreign key references a primary key in another table, enforcing referential integrity between them.

Q How do you use SQL in your QA work?

A To validate that UI/API actions produced correct database state — verifying inserted/updated records, checking constraints, and cross-checking report figures. I also automated reporting with SQL to cut manual processing time.

Q WHERE vs HAVING?

A WHERE filters rows before grouping; HAVING filters groups after GROUP BY aggregation. You use HAVING to filter on aggregate results like COUNT/SUM.

18. Rapid-Fire Viva Question Bank

Short, high-frequency questions to drill the day before an interview.

Q What is a test bed / test environment?

A The hardware, software, network and data configuration on which tests run, set up to mirror production as closely as possible.

Q What is positive vs negative testing?

A Positive testing feeds valid inputs to confirm expected behavior; negative testing feeds invalid/unexpected inputs to confirm the system handles errors gracefully.

Q What is a test harness?

A The collection of tools, stubs, drivers and configuration that lets a test suite run automatically.

Q What is defect leakage / defect density?

A Defect leakage = defects that escaped to the next phase/production (missed by QA).
Defect density = defects per size unit (e.g., per KLOC or per module) — a quality indicator.

Q What is a test closure report?

A The end-of-cycle summary: what was tested, pass/fail metrics, open defects, coverage, and recommendations for release readiness.

Q What is risk-based testing?

A Prioritizing test effort on the features with the highest probability and impact of failure, so limited time is spent where it matters most.

Q What is the cost of fixing a bug over time?

A It rises sharply the later it's found — cheapest in requirements, most expensive in production — which justifies shift-left testing.

Q What is a blocker bug?

A A defect that stops further testing of a feature or the whole build until it's fixed — highest priority.

Q What is the 80/20 (Pareto) principle in testing?

A Roughly 80% of defects come from 20% of modules; focus testing on those defect-prone areas.

Q What are the seven testing principles?

A Testing shows presence (not absence) of defects; exhaustive testing is impossible; early testing; defect clustering; pesticide paradox (repeated tests stop finding new bugs); testing is context-dependent; absence-of-errors fallacy (a bug-free but useless product still fails).

Q What is the pesticide paradox and how do you fight it?

A Re-running the same tests eventually stops finding new bugs. I counter it by regularly reviewing and adding new/varied test cases and doing exploratory testing.

Q Why can't we test everything (exhaustive testing)?

A The number of input combinations, paths and states is effectively infinite; instead we use risk-based prioritization and design techniques (EP, BVA, decision tables) to get high coverage efficiently.

Q What is your biggest strength as an SQA engineer?

A Accuracy-first, detail-oriented testing — I sustain ~98% defect reproduction accuracy, write clear reproducible reports that reduce re-open rates, and combine manual depth with automation that cut regression effort ~35%.

Q How do you decide what to automate?

A Automate stable, repetitive, high-value, frequently-run tests (regression, smoke, data-driven API checks); keep exploratory, usability, one-off and rapidly changing areas manual.

Interview tip: for every tool on your CV, be ready to say (1) what it is, (2) why you chose it, (3) a concrete example from your projects, and (4) one challenge you solved with it. That four-part story is what separates a memorized answer from a credible engineer.